CE-2103 · Lección 01

Administración de Memoria

Cómo se organiza y se usa la memoria de un programa



Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

¿Qué aprenderemos hoy?

1. Explicar el papel del sistema operativo en la administración de memoria
2. Describir cómo evolucionó la memoria: espacios de direcciones y memoria virtual
3. Reconocer las secciones del memory layout de un programa en C
4. Elegir entre stack y heap y reconocer sus riesgos (fugas, fragmentación)
5. Usar punteros: `&`, `*`, sharing, copias y aritmética
6. Comparar punteros con las referencias de lenguajes manejados y de Rust

// sistema operativo

El sistema operativo como API

Ejemplo: abrir un archivo con la API del SO

```
FILE* fptr = fopen("archivo.txt", "r");
```

`fopen` retorna un pointer (handler) al archivo sin que usted conozca cómo el SO lo implementa.

Sistema operativo

Capa de software que interactúa directamente con el hardware. Es el intermediario entre el hardware y las aplicaciones.

API

Application Programming Interface: una capa de abstracción para usar un sistema sin conocer su implementación.

POSIX

Portable Operating System Interface: estándar del IEEE para las API del sistema operativo.

// sistema operativo

El SO administra los recursos de la máquina

El hardware es complejo; el SO se encarga de él. Recursos que administra:

CPU

Ejecuta las instrucciones de los procesos.

Memoria principal

Aloja los programas y datos en ejecución.

Discos

Almacenan los archivos de forma permanente.

I/O

Entrada y salida: dispositivos y periféricos.

> El sistema operativo **hace transparentes** los recursos de la máquina: los desarrolladores no lidian con el hardware directamente.

// sistema operativo

Programa, proceso y thread

Tres conceptos que no deben confundirse:

Programa

Archivo ejecutable en disco con el código: el conjunto de instrucciones del procesador.

Proceso

Programa cargado en memoria, con los recursos del SO que necesita: registros, contador de programa, punteros de pila, páginas.

Thread (hilo)

Flujo de control dentro de un proceso, con su propia pila de llamadas, registros y estado. Comparte la memoria del proceso.

> Los threads de un proceso comparten el **espacio de direcciones** y los archivos abiertos.

// administración de memoria

Responsabilidades del sistema operativo

La administración de memoria es el conjunto de tareas con que el SO gestiona la memoria principal:

1

Asignar

Asignar y liberar memoria al inicio y al fin de un proceso.

2

Controlar

Controlar la memoria asignada a cada proceso.

3

Fragmentación

Minimizar la fragmentación.

4

Integridad

Mantener la integridad de los datos.

Límite y aislamiento: controlar la memoria de cada proceso evita que afecte a otros procesos en ejecución.

Analogía: el edificio de apartamentos

Edificio de apartamentos	Memoria de la computadora
Apartamentos numerados consecutivamente	Direcciones de memoria consecutivas
Cada apartamento aloja a una persona	Cada dirección aloja 1 byte
Una familia ocupa apartamentos contiguos	Una variable ocupa 1 o más bytes contiguos
La recepción lleva la correspondencia	El sistema operativo genera las direcciones
La constructora agrega pisos	El usuario instala más RAM

> Cada dirección identifica **un byte**; una variable ocupa uno o más bytes **contiguos**.

// direcciones de memoria

Direcciones de n bits y memoria accesible

Bits	Cantidad de direcciones	Rango	Memoria accesible
16 bits	65 536	0 a 65 535	64 KB
32 bits	4 294 967 296	0 a 4 294 967 295	4 GB
n bits	2^n	0 a 2^n – 1	2^n bytes

Ojo: con direcciones de 32 bits y 8 GB de RAM, el SO solo accede a 4 GB: no tiene direcciones para más.

Típico: los sistemas operativos usan direcciones de 32 o 64 bits.

// evolución de la memoria

Sin abstracción de memoria

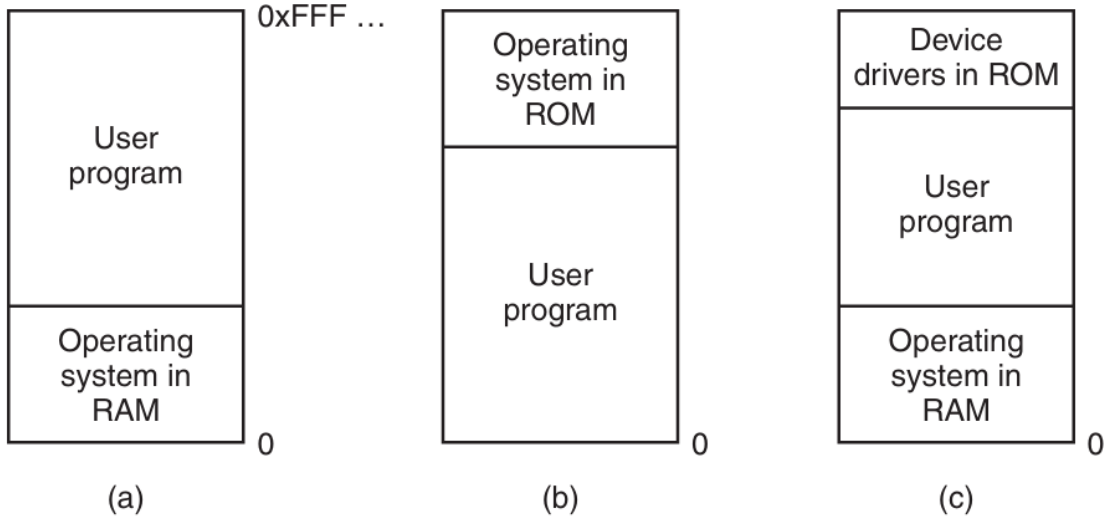
```
MOV REGISTER1, 1000
```

Memoria física

Mainframes (años 60), minicomputadoras (70) y PC (80): el programa accede directo a la memoria **física**. La dirección 1000 es la 1000 real.

Sin multiprogramación

Dos programas cargados a la vez podían afectarse entre sí y causar errores.



(a) SO en RAM · (b) SO en ROM · (c) drivers en ROM

// evolución de la memoria

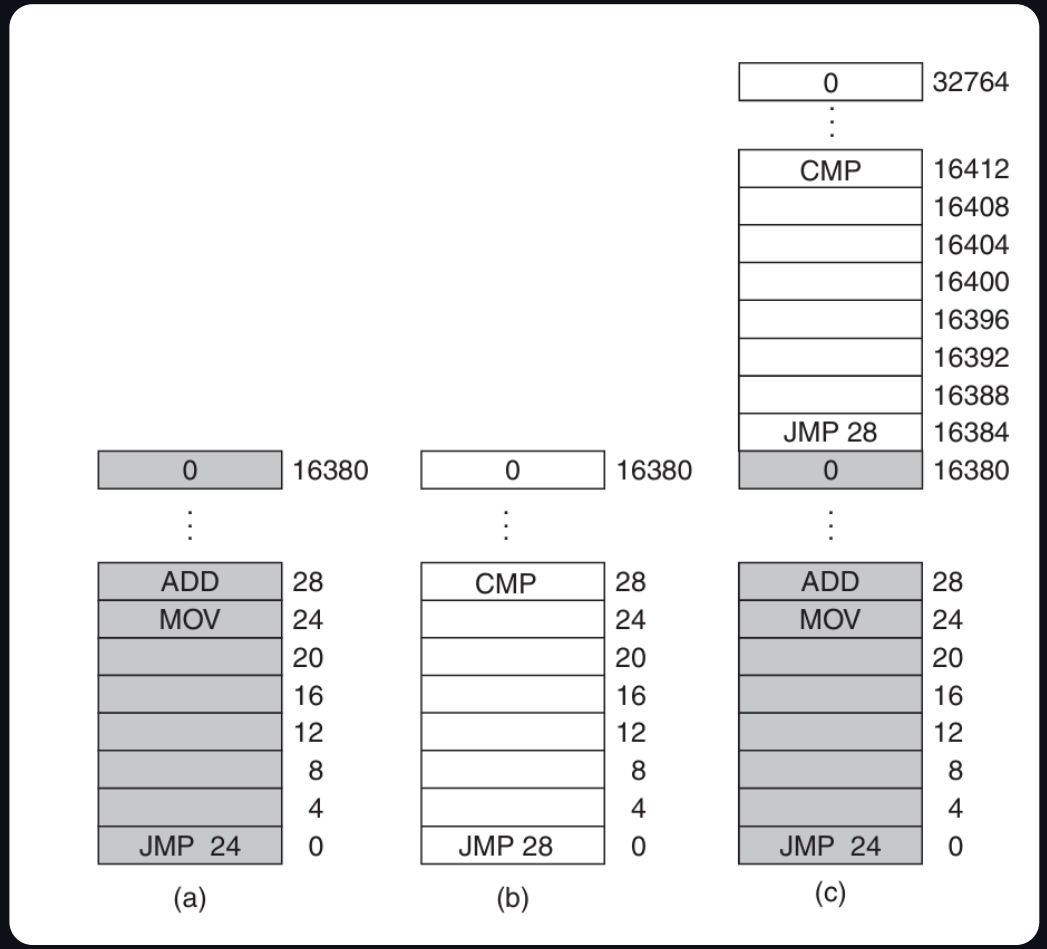
El problema de la re-ubicación

El problema

En (c) el segundo programa se carga en 16 384, pero su `JMP 28` salta a una dirección del primer programa.

La causa

Las direcciones del programa asumen que empieza en 0; al cargarlo en otra posición dejan de ser válidas.



Swapping: pausar un programa y cargar otro

1

Pausar

Se pausa la ejecución del programa actual.

2

Guardar

Su estado se guarda en el disco.

3

Cargar otro

Se carga otro programa en memoria y se ejecuta.

A cierto grado: solo un programa está en memoria a la vez; mientras uno espera una operación de I/O, otro puede ejecutarse.

// evolución de la memoria

Static relocation (IBM 360)

Programa en disco

Las referencias asumen que el programa empieza en 0:

```
JMP 28
```

Valores ilustrativos tomados de la figura anterior.

Al cargarlo con base 16 384

Se suma la dirección base a cada referencia:

```
JMP 16412    // 28 + 16 384
```

Limitación: no es eficiente; entre más grande el programa, más tarda en cargarse.

Monoprogramación vs. multiprogramación

Monoprogramación

- Un proceso a la vez
- Si espera una operación de E/S, la CPU queda **ociosa**

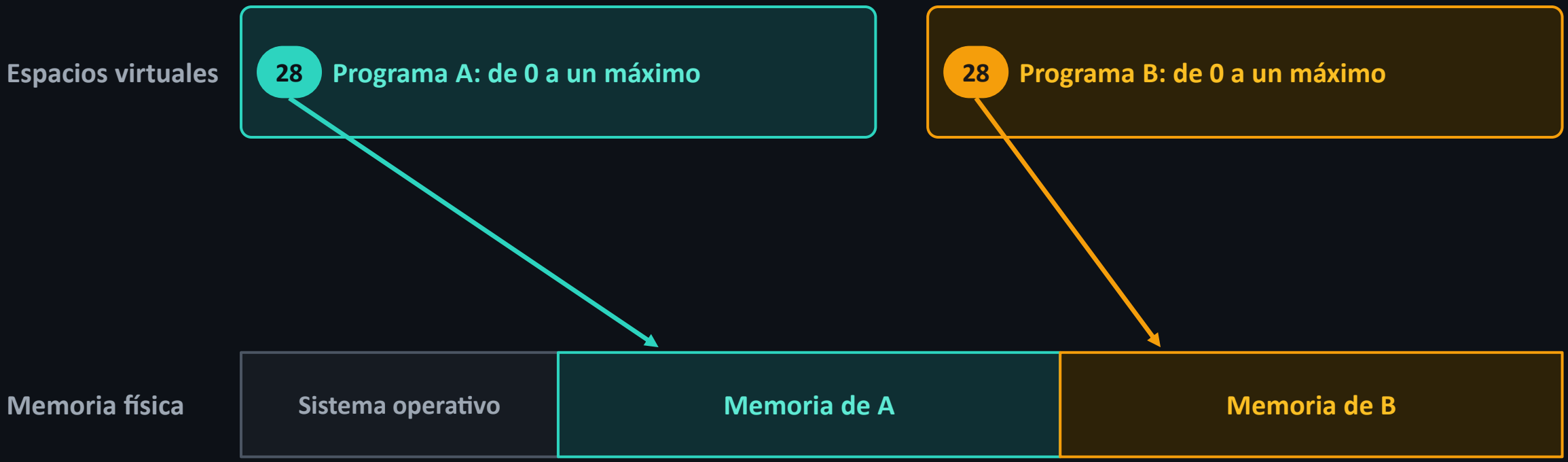
vs

Multiprogramación

- Dos o más procesos en memoria principal
- Se ejecutan de forma **concurrente**, solapados en el tiempo
- Dan la impresión de ejecución en paralelo

Concurrencia vs. paralelismo: concurrente = tareas traslapadas (basta un núcleo); paralelo = al mismo tiempo (varios núcleos).

Espacios de direcciones

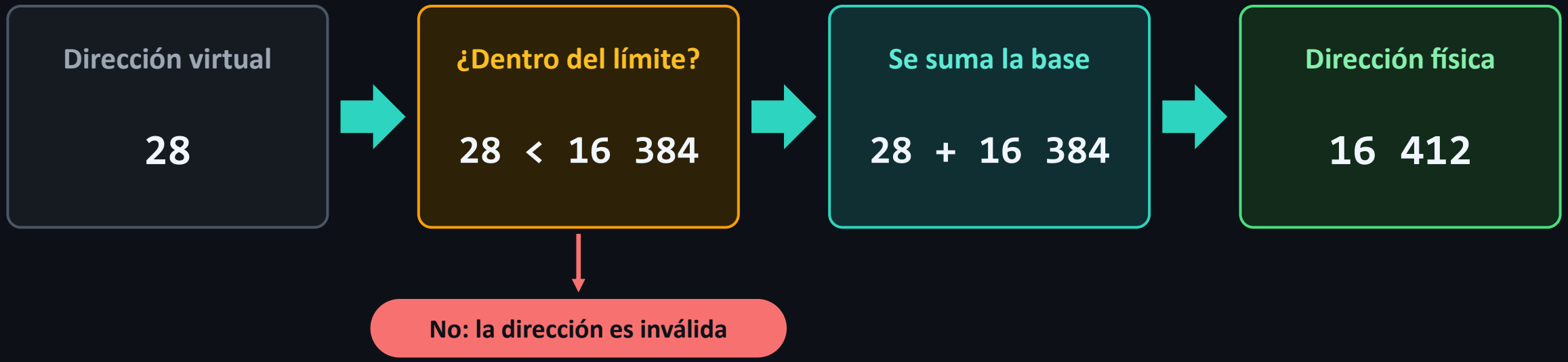


Dos programas pueden ver la dirección 28, pero esta se refiere a direcciones **físicas diferentes**.

// evolución de la memoria

Registros base y límite

El SO los carga al ejecutar el programa: **base = 16 384** **límite = 16 384**



Limitante: el programa completo debe caber en la memoria, poco práctico para programas modernos.

// evolución de la memoria

Memoria virtual: páginas y frames

Solución para ejecutar programas que no caben en la memoria física:

1

Páginas

Bloques contiguos en que se divide el espacio de direcciones de un programa.

2

Frames

Bloques de memoria física a los que se mapean las páginas.

3

No todas cargadas

Solo algunas están en memoria; las demás se cargan del disco al necesitarlas.

En `MOV REGISTER1, 1000`, la dirección 1000 es una dirección **virtual** de su espacio de direcciones virtuales.

Page-hit vs. page-fault

Page-hit

- La página referenciada **está cargada** en un frame
- El hardware mapea la dirección virtual a la dirección física correspondiente

vs

Page-fault

- La página **no está cargada**
- El SO la carga desde el disco a un frame libre
- Se re-ejecuta la instrucción que causó el fallo

> Si no hay frames libres, un **algoritmo de reemplazo de páginas** decide cuál página sacar.

El MMU: de virtual a física

MMU

Memory Management Unit: hardware especializado que mapea las direcciones virtuales a direcciones físicas.

El flujo

La CPU envía direcciones virtuales al MMU; el MMU envía direcciones físicas a la memoria.

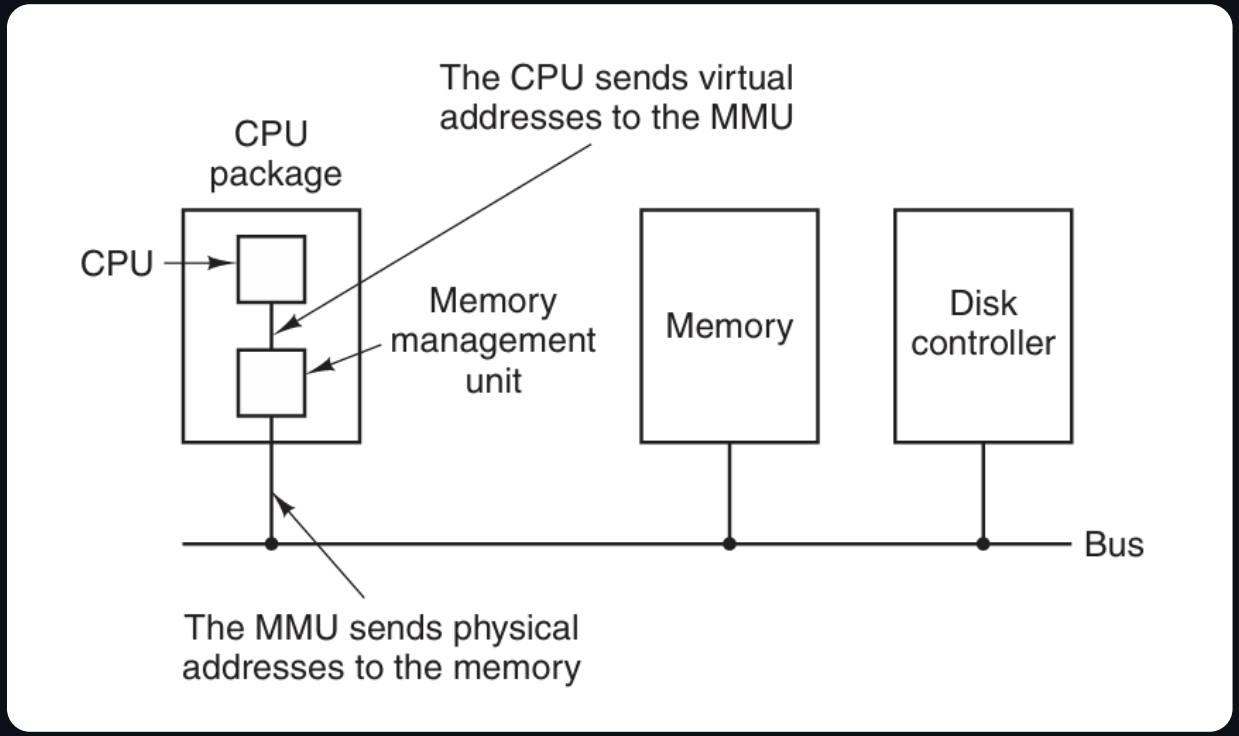


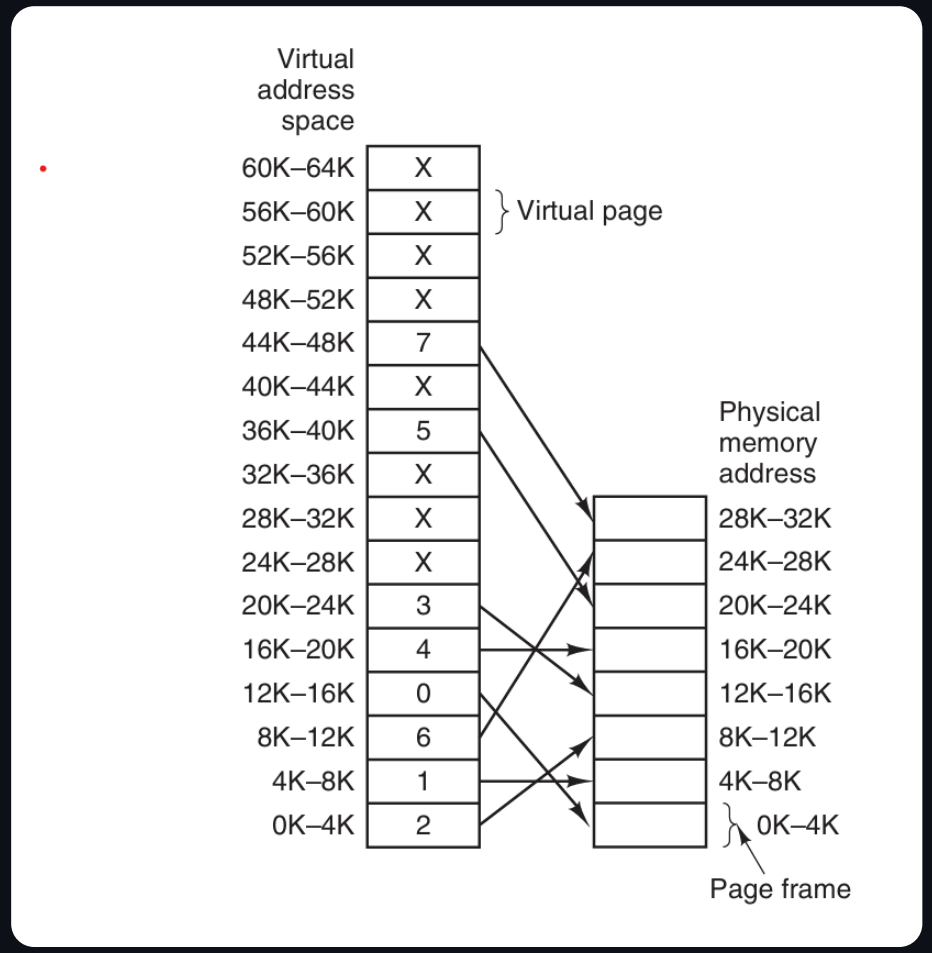
Tabla de páginas y reemplazo de páginas

Tabla de páginas

Lleva el inventario de páginas cargadas y su frame, con estadísticas como la frecuencia de uso. «X» = página no cargada.

Reemplazo de páginas

Como los frames son limitados, un algoritmo decide qué página sacar para cargar otra.



// memoria de un proceso

¿Por qué memoria en estructuras de datos?

Hasta aquí vimos la memoria desde el sistema operativo; desde aquí, desde el programa.

Viven en el heap

Listas enlazadas y árboles crecen dinámicamente y se enlazan con punteros.

La memoria importa

La estructura más eficiente se echa a perder si la memoria se maneja mal.

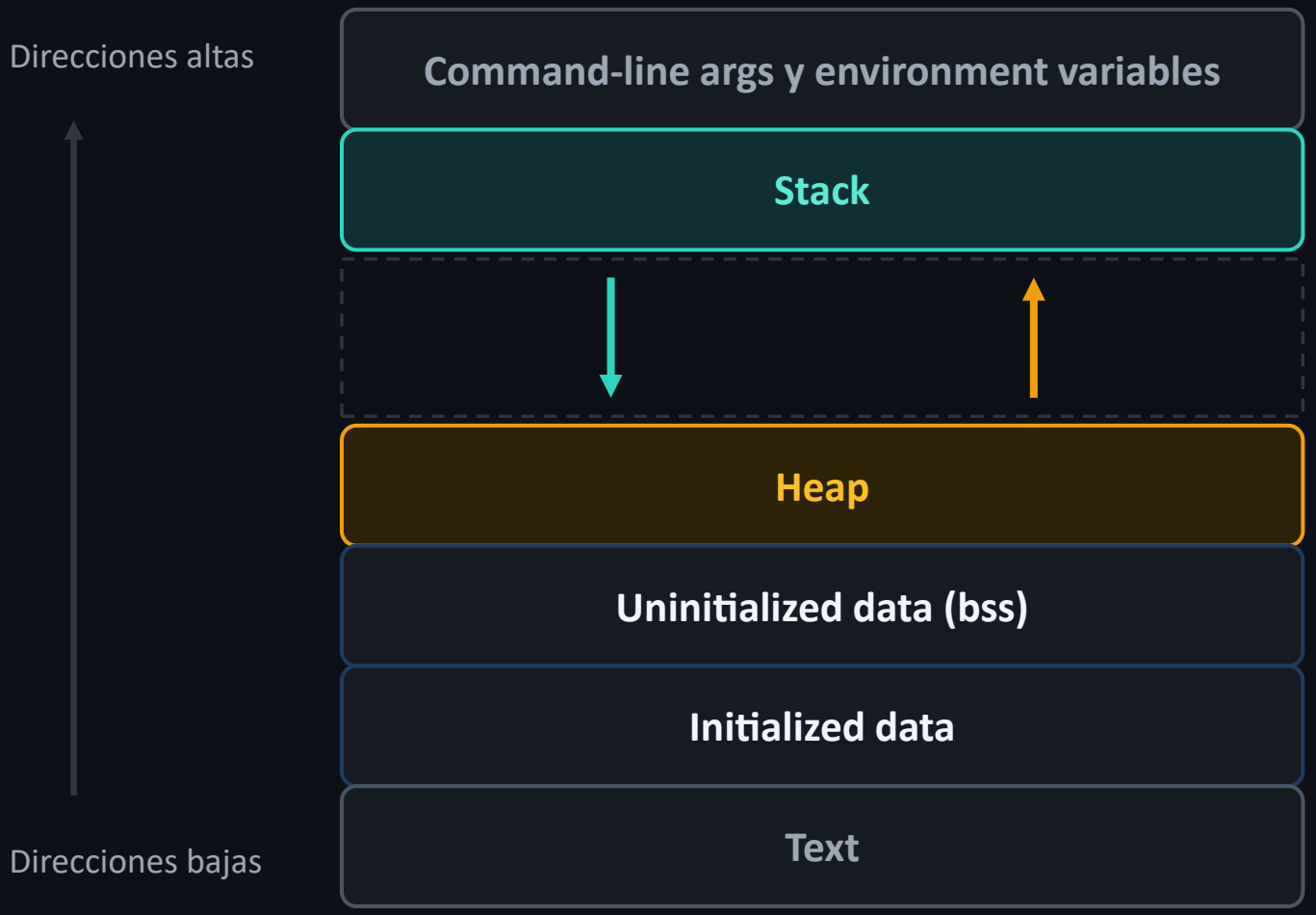
Errores a evitar

Fugas de memoria, punteros colgantes y acceso a memoria ya liberada.

Para construir buenas estructuras de datos hay que entender **cómo se organiza y se usa la memoria** de un programa.

// memoria de un proceso

Memory layout de un programa en C



Argumentos del programa y variables de entorno

Variables locales y stack frames · crece hacia abajo

Espacio libre entre ambos

Memoria dinámica (malloc, new) · crece hacia arriba

Globales y estáticas sin inicializar

Globales y estáticas inicializadas

Código ejecutable (solo lectura)

// memoria de un proceso

Secciones del memory layout

Sección	Qué contiene	Acceso o gestión
Text	Código ejecutable; compartido entre procesos del mismo programa	Solo lectura
Initialized data	Variables globales y estáticas inicializadas	Lectura y escritura
Uninitialized data (bss)	Globales y estáticas sin inicializar; el compilador las pone en cero	Lectura y escritura
Command-line args y env	Argumentos de la línea de comandos y variables de entorno	Se cargan al ejecutar
Stack	Variables locales, argumentos y return address de cada llamada	Automática (LIFO)
Heap	Datos de tamaño desconocido al compilar, de vida controlada por el programador	Manual (malloc y free)

// memoria de un proceso

Initialized data: globales y estáticas

Segmento de datos (variables globales)

```
int x = 10;           // read-write
const int y = 20;     // read-only
char s[] = "hola";    // read-write

int main() {
  // Código principal
}
```

Dos áreas

Una para variables read-only y otra para read-write.

¿static en C?

En una función, el valor persiste entre llamadas. En ámbito global, el elemento solo existe en su unidad de compilación.

> `x`, `y` y `s` son variables **globales**: viven en el segmento de datos, no en el stack.

Uninitialized data (bss)

Variables sin inicializar

```
int x;  // uninitialized
int y;  // uninitialized

int main() {
  // Código principal

  static int z; // uninitialized
}
```

bss

Block started by symbol: segmento de datos sin inicializar.

En cero

El compilador las inicializa en cero antes de que el programa empiece a ejecutarse.

> Incluso `static int z;`, declarada dentro de `main`, vive en este segmento.

// memoria de un proceso

Argumentos y variables de entorno

Se ejecuta con argumentos

```
$ ./programa -a -b -c
```

En bash se define una variable de entorno

```
export MYVAR="Ejemplo de variable"
```

Se accede con argc y argv

```c
int main(int argc, char* argv[]) {
  // argc: número de argumentos
  // argv[0]: nombre del programa
  // argv[1]: primer argumento
  printf("Arg 1: %s\n", argv[1]);
}
```

Aquí `argc` vale 4 y `argv[1]` es `-a`.

Se accede con environ

```c
extern char** environ;

int main() {
  for (int i = 0; environ[i] != NULL; i++) {
    printf("%s\n", environ[i]);
  }
}
```

// memoria de un proceso

Stack: la memoria automática

Almacena las variables locales y los argumentos de las funciones.

1

LIFO

Last In, First Out: la última función llamada es la primera en terminar.

2

Stack frame

Cada llamada crea un frame con las variables locales, los argumentos y el return address.

3

Automático

Al terminar la función su frame se elimina: por eso las variables locales son «automáticas».

Las variables del stack deben tener **tamaño conocido al compilar**: una lista enlazada no puede almacenarse aquí.

El stack en acción

```c
int main(void) {
  printf("hello world\n");
  first_function();
  printf("goodbye goodbye\n");
  return 0;
}
void first_function(void) {
  int imidate = 3;
  char broiled = 'c';
  void *where_prohibited = NULL;
  second_function(imidate);
  imidate = 10;
}
void second_function(int a) {
  int b = a;
}
```

Stack al llamar a second_function

main

first_function

Return address: main

int imidate

char broiled

void* where_prohibited

second_function

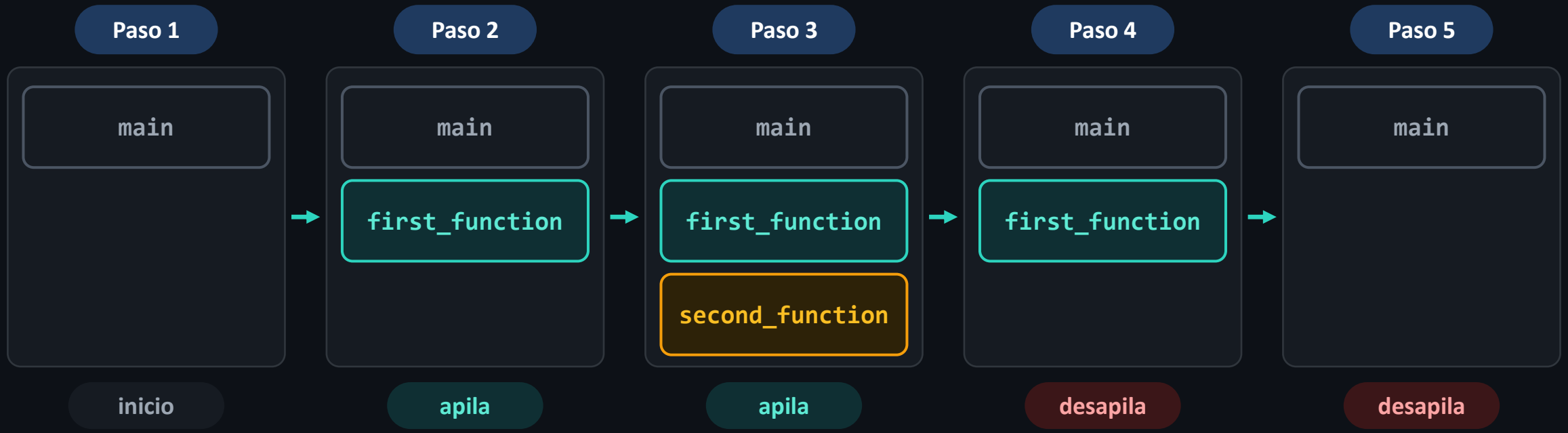
Return address: first_function

int a (parámetro)

int b

// memoria de un proceso

El stack crece y decrece con las llamadas



Cada llamada **apila** un frame; cada retorno lo **elimina**. El último en entrar es el primero en salir (LIFO).

¿Y la recursión? Cada llamada recursiva apila su propio frame; sin caso base el stack se agota (**stack overflow**).

// memoria de un proceso

Heap: la memoria dinámica

Para datos cuyo tamaño no se conoce al compilar y cuyo tiempo de vida controla el programador:

Tamaño variable

Se expande y contrae dinámicamente: listas enlazadas, árboles, etc.

Control manual

El programador asigna, des-asigna y re-dimensiona la memoria.

API del heap

C: `malloc` (sin inicializar; retorna `NULL` si falla), `calloc` (en cero), `realloc`, `free`. C++: `new` y `delete`.

> **¿Se puede evitar el heap?** Sí, pero solo con variables de tamaño conocido al compilar: menos flexibilidad.

// memoria de un proceso

El heap en acción: malloc y free

```c
int *age = malloc(sizeof(int));
*age = 30;
double *salary = malloc(sizeof(double));
*salary = 12345.67;
double *myList = malloc(3 * sizeof(double));
// myList[0..2] = 1.2, 2.3, 3.4
double *twiceSalary = multiplyByTwo(salary);
printf("%.3f\n", *twiceSalary);
free(age);
free(salary);
free(myList);
free(twiceSalary);
```

Tras los malloc

0x0088
double (twiceSalary)

0x0040
3 × double (myList)

0x0020
double (salary)

0x0000
int (age)

Tras los free

Todos los bloques fueron liberados

Cada `malloc` necesita su `free`: al liberar todos los bloques, el heap queda vacío. Las direcciones son ilustrativas.

// memoria de un proceso

Sin puntero, la memoria se pierde

Con puntero

El puntero (en el stack) guarda la dirección `0x64`:

```
int* p = malloc(sizeof(int));
```

La memoria es accesible y se puede liberar con `free(p)`.

vs

Sin puntero

Ningún puntero guarda la dirección `0x64`:

```
malloc(sizeof(int));
```

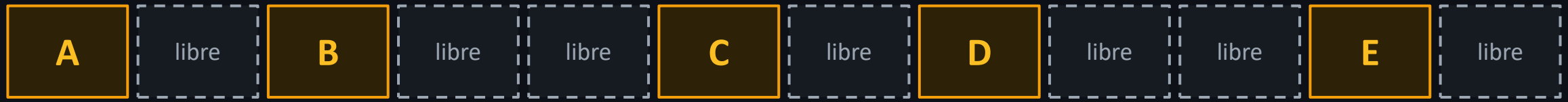
La memoria es inaccesible: fuga de memoria (memory leak).

El puntero vive en el stack y se libera con su frame, pero la memoria del heap **no se libera sola**.

Fragmentación: memoria libre que no sirve

Tras varios malloc y free

Pedido de 3 celdas contiguas: no cabe



Hay 7 celdas libres, pero nunca 3 seguidas

Compactación: se mueven los bloques ocupados para juntar los huecos (operación **costosa**)

Ocupado Libre Pedido atendido

Después de compactar

Pedido de 3 celdas contiguas: sí cabe



Las 7 celdas libres quedan juntas: cabe el pedido y sobran 4

Fragmentación: hay memoria libre suficiente en total, pero **dispersa**, y no sirve para un bloque grande contiguo.

Matiz: en C el heap no se compacta (los punteros dejarían de ser válidos); lo hacen el SO o el recolector de basura.

// memoria de un proceso

Heap vs. stack: ¿cuál usar?

	Heap	Stack
Tipo	Memoria dinámica	Memoria automática
Tamaño	Variable	Fijo
Tiempo de vida	Lo controla el programador	Termina con la función
Responsable	El programador	El programador no es responsable
Transparencia	No es transparente	Transparente
Errores	Propenso a bugs del programador	Menos propenso (gestión automática)

Use el heap si necesita bloques grandes, datos que vivan más allá de la función o estructuras que crezcan.

Use el stack si son variables pequeñas, de tamaño conocido, que solo se usan en su ámbito.

// punteros

Punteros: un tipo de dato especial

Parte del API del heap. Como todo tipo de datos, define sus valores y sus operaciones:

Valores

Su rango de valores son direcciones de memoria.

Operaciones

Asignación, des-asignación, aritmética de punteros y acceso a memoria.

¿Por qué con tipo?

El compilador verifica tipos y sabe cuántos bytes leer, escribir o sumar.

> No hay nada «mágico»: un puntero es un dato cuyo valor es una **dirección de memoria**.

// punteros

Declarar punteros y reservar memoria

Código

```
int* ptr = malloc(sizeof(int));
char* cptr = malloc(sizeof(char));
double* dptr = malloc(sizeof(double));
```

Tabla de memoria

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	ptr	**0x64**	4B	**Stack**	Pointer
0x4	cptr	**0x68**	4B	**Stack**	Pointer
0x8	dptr	**0x69**	4B	**Stack**	Pointer
0x64		**?**	4B	**Heap**	int
0x68		**?**	1B	**Heap**	char
0x69		**?**	8B	**Heap**	double

El puntero vive en el **stack**; la memoria de `malloc`, en el **heap** y **sin inicializar** (`?` = basura; `calloc` la pone en cero). Direcciones ficticias.

// punteros

Operador address-of (&)

Código

```
int x = 10;
int* ptr = &x;

cout << ptr;
// imprime, p. ej., 0x7fffbf7b3b7c
```

Tabla de memoria

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	**10**	4B	**Stack**	int
0x4	ptr	**0x0**	4B	**Stack**	Pointer

> `&x` obtiene la **dirección** de `x` (no confundir con el AND de bits `&`). Sirve para memoria del stack o del heap.

// punteros

Operador de indirección (*)

Código

```
int x = 3899;
int* price = &x;

cout << *price; // 3899
*price = 1000;
cout << x;      // 1000
```

Antes de *price = 1000;

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	**3899**	4B	**Stack**	int
0x4	price	**0x0**	4B	**Stack**	Pointer

Después de *price = 1000;

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	**1000**	4B	**Stack**	int
0x4	price	**0x0**	4B	**Stack**	Pointer

> `*price` accede al valor apuntado, y también permite **modificarlo**.

// punteros

Sharing: dos punteros, una dirección

Código

```
int x = 20;
int* ptr = &x;
int* ptr2 = ptr;

*ptr = 30;
// x, *ptr y *ptr2 valen 30
```

Antes de *ptr = 30;

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	**20**	4B	**Stack**	int
0x4	ptr	**0x0**	4B	**Stack**	Pointer
0x8	ptr2	**0x0**	4B	**Stack**	Pointer

> Con **cualquiera** de los punteros se lee o modifica x: ambos apuntan a la misma dirección.

Shallow-copy vs. deep-copy

Shallow-copy

```
int* counterPtr = malloc(sizeof(int));
*counterPtr = 5;
int* counterPtr_2 = counterPtr;
```



Se copia el puntero: un solo valor

Deep-copy

```
int* counterPtr_2 = malloc(sizeof(int));
*counterPtr_2 = *counterPtr;
```



Se copia el valor: dos espacios

Si una estructura tiene campos puntero, la deep-copy debe copiar el valor al que apunta cada puntero, no el puntero.

Traza: valor, dirección y puntero a puntero

Código

```
int data = 5;
int* pointer = &data;
int* pointer2 = pointer;
int data2 = data;
int data3 = *pointer2;
int** pointerPointer = &pointer;

data = 8;
```

Memoria (stack) antes y después de `data = 8;`

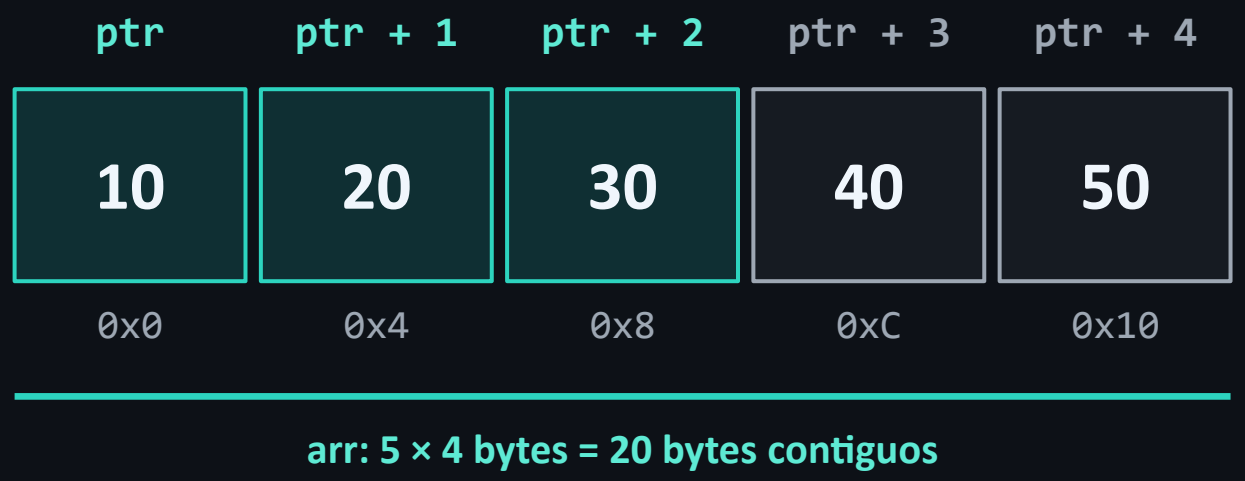
Dirección	Alias	Antes	Después	Tipo
0x0	data	**5**	**8**	int
0x4	pointer	**0x0**	**0x0**	int*
0x8	pointer2	**0x0**	**0x0**	int*
0xC	data2	**5**	**5**	int
0x10	data3	**5**	**5**	int
0x14	pointerPointer	**0x4**	**0x4**	int**

`data2` y `data3` son **copias del valor**: siguen en 5. `*pointer`, `*pointer2` y `**pointerPointer` ven el 8, porque llevan a `data`.

Aritmética de punteros

```
int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr;

cout << *ptr;        // 10
cout << *(ptr + 1); // 20
cout << *(ptr + 2); // 30
```



`ptr + 1` avanza **un elemento** (4 bytes para `int`), no un byte. Además `arr[i]` equivale a `*(arr + i)`: acceso **O(1)**.

// punteros

Paso de parámetros: valor vs. referencia

Por valor

Se pasa una **copia**: la variable original no cambia.

```
void foo(int x) {
  x = 20;
}
// con x = 10, foo(x) la deja en 10
```

vs

Por referencia

Se pasa la **dirección**: la variable original sí cambia.

```
void foo(int& x) {
  x = 20;
}
// con x = 10, foo(x) la deja en 20
```

En C: `void foo(int* x) { *x = 20; }` y se llama con `foo(&x)`. En C++, `int&` es un alias: no puede ser nulo ni reasignarse.

Buenas prácticas con punteros

- **Inicialice y verifique los punteros**
  Un puntero sin inicializar tiene un valor aleatorio: compárelo con `NULL` antes de usarlo.
- **Libere la memoria dinámica**
  Cuando ya no la necesite, para evitar fugas de memoria.
- **Evite los punteros colgantes**
  Después de `free`, asigne `NULL` al puntero.
- **Use const cuando sea posible**
  Si el puntero no debe modificar los datos a los que apunta.
- **Evite la aritmética compleja**
  Es propensa a errores: úsela con cuidado.
- **No retorne punteros al stack**
  La variable se libera al terminar la función.

// punteros

Inicializar, verificar y liberar

Inicializar

```
int* ptr = NULL; // Correcto
int* ptr;        // Incorrecto
```

Liberar y evitar punteros colgantes

```
free(ptr);
ptr = NULL;
```

Verificar antes de usar

```
if (ptr != NULL) {
  // Usar el puntero
}
```

const y aritmética sencilla

```
const int* ptr = &x;

int arr[10];
int* p = arr;
p += 2; // tercer elemento
```

Un puntero sin inicializar puede causar errores difíciles de depurar o un **segmentation fault** al de-referenciarlo.

// punteros

No retornar punteros a variables del stack

Comportamiento indefinido

```
int* foo() {
  int temp = 50;
  return &temp;
}
void bar() {
  int temp = 66;
  return;
}
int main() {
  int* r = foo();
  cout << *r; // ¿?
  bar();
  cout << *r; // ¿?
}
```

Puntero colgante

`temp` vive en el frame de `foo`, que se libera al retornar. `r` apunta a memoria ya liberada.

Salida no confiable

La memoria pudo reutilizarse (por `bar`), pero no está garantizado: puede imprimir cualquier valor o fallar.

Referencias en lenguajes manejados

Java

```
MyClass obj = new MyClass();
obj.x = 10;

MyClass obj2 = obj; // Shallow-Copy
System.out.println(obj.x); // 10
```

Referencia

En Java, C# y Python no hay punteros: un identificador apunta a un objeto.

Distinta de C++

Puede ser nula (`null`, `None`) y reasignarse a otro objeto.

`obj2 = obj` copia la **referencia**, no el objeto: ambas variables apuntan al mismo objeto (shallow-copy).

// lenguajes manejados

¿Hay fuga de memoria? C vs. Java

C: sí hay fuga

Al reasignar b se pierde la dirección de su bloque:

```
int* a = malloc(sizeof(int));
int* b = malloc(sizeof(int));
b = a; // 2.º bloque sin puntero
free(a);
```

vs

Java: no hay fuga

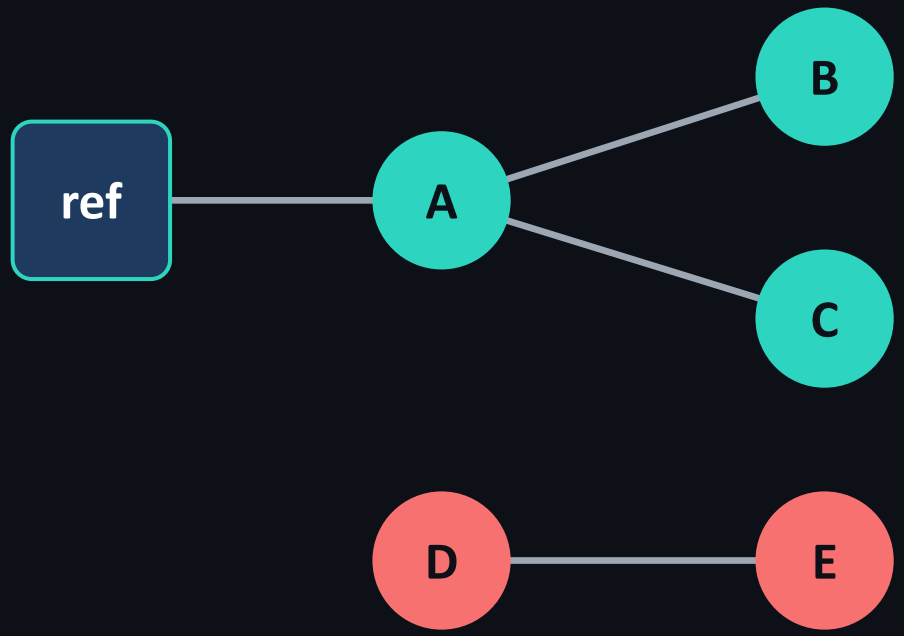
El objeto que b dejó de referenciar es inaccesible:

```
Foo a = new Foo();
Foo b = new Foo();
b = a; // el 2.º objeto es inaccesible
```

El recolector de basura lo libera por usted.

Compromiso: Java evita estos bugs, pero el recolector de basura puede hacer más lento el programa; en C usted controla la memoria y es responsable de ella.

Recolección de basura: marcado y barrido



Objetos	Resultado
A, B, C	Accesibles desde la referencia: se conservan
D, E	Inaccesibles: se marcan para liberar

El recolector **sigue las referencias** entre objetos; la memoria de lo que no alcanza se libera automáticamente.

Rust: ownership y borrowing

Sí compila

Varias referencias **inmutables** a la vez: válido.

```
fn main() {
    let mut x = 10;
    let y = &x;
    let z = &x;
    println!("{}", x);
}
```

vs

No compila

Una referencia **mutable** con otra coexistiendo.

```
fn main() {
    let mut x = 10;
    let y = &mut x;
    let z = &x; // error
    println!("{} {}", y, z);
}
```

Regla: a lo sumo **una** referencia mutable, **o** cualquier número de inmutables; nunca ambas a la vez.

// práctica

Ahora usted: resuelva cada caso

A · Punteros

Con `int a = 5; int* p = &a; *p = *p + 1;`, ¿cuánto vale `a`?

B · Layout

¿En qué sección vive una variable local, un bloque de `malloc`, una global inicializada y una global sin inicializar?

C · Encuentre el error

¿Qué falla en `int* f() { int t = 7; return &t; }`?

D · Direcciones

Un SO de 32 bits con 8 GB de RAM instalada: ¿cuánta memoria puede acceder?

E · Swap

Escriba `swap(int*, int*)` para intercambiar dos enteros. ¿Por qué no basta pasarlos por valor?

F · Struct y ->

Si `f` apunta a un `struct fraction` del heap, escriba dos formas de asignar `22` a `numerator`.

// cierre

Resumen

- **Sistema operativo:** hace transparentes los recursos y administra la memoria principal.
- **Evolución:** sin abstracción, espacios de direcciones y memoria virtual (páginas, frames, MMU).
- **Memory layout:** text, initialized data, bss, args y env, stack y heap.
- **Stack vs. heap:** automático y fijo, o dinámico y manual; fugas y fragmentación.
- **Punteros:** &, *, sharing, copias, puntero a puntero, aritmética y buenas prácticas.
- **Lenguajes manejados:** referencias y recolección de basura; Rust: ownership y borrowing.

Referencias: Tanenbaum, A. S. y Bos, H. (2014). Modern Operating Systems. Pearson · geeksforgeeks.org (memory-layout-of-c-program, cpp-pointer-operators) · Lectura complementaria: Parlante, N. Pointers and Memory. Stanford CS Education Library, doc. n.º 102